

교과목 3 : 초거대언어모델(LLM)

단원 2 : 자연어 딥러닝

DAY4. 자연어 처리를 위한 번역 모델 구조

목 차

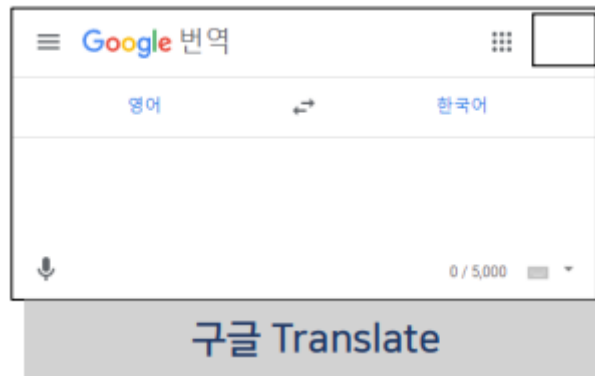
1. 자동 번역기 기술		3
2. 구글 AI 번역 기술		14
3. Seq2Seq		21
4. Seq2Seq RNN 스마트 번역기		16

1. 자동 번역기 기술

1) 한영 번역기 대결

한영 번역기 중 어떤 서비스를 사용해 보셨나요?

- Google Translate (구글 번역)
- 네이버 파파고



- **Google Translate** 의 번역 화면입니다.
 - 왼쪽 입력창에 원문을 입력하면 오른쪽에 번역 결과가 출력되는 구조입니다.
 - 영어와 한국어를 서로 번역할 수 있는 인터페이스입니다.



- **네이버 Papago** 화면입니다.
 - 입력창에 문장을 입력하면 번역 결과를 제공합니다.
 - 음성 입력과 번역 실행 버튼도 함께 표시되어 있습니다.

자동 번역 기술은 현재 일상생활에서 매우 많이 활용되고 있으며, 대표적인 서비스로 **구글 번역기**와 **네이버 파파고**가 있습니다.

(2021 년 12 월 기준)

한국어	구글	파파고
영문을 모르겠어.	I don't know English.	I don't know English.
씨가 말랐다.	The seeds are dry.	The seeds have dried up.
어딜 도망가?	Where are you running?	Where are you going?
뭐하노?	What are you doing?	What are you doing?
통촉하여 주시옵소서.	Please contact me.	Please let me know.

정답 : I don't know the reason.

문장은 '영문을 모르겠어' 입니다.

두 번역기 모두 'I don't know English.' 라고 번역했습니다.

그러나 실제 의미는 '영어를 모른다' 가 아니라 '이유를 모르겠다' 라는 의미입니다.

올바른 번역은 'I don't know the reason.' 입니다.

번역기는 단어를 그대로 해석하는 경우가 많아 문맥을 이해하지 못하면 의미가 달라질 수 있습니다.

'씨가 말랐다.'

구글 : The seeds are dry.

파파고 : The seeds have dried up.

정답 : Nothing is left.

한국어에서는 "씨가 말랐다" 라는 표현이

- 씨앗이 말랐다

라는 의미가 아니라

- 모두 없어졌다.
- 흔적도 남지 않았다.

라는 관용적인 의미로도 사용됩니다.

구글과 파파고 모두 씨(seed)를 문자 그대로 해석했습니다.

따라서 올바른 번역은 'Nothing is left.' 입니다.

번역기는 **관용 표현(idiom)** 을 이해하지 못하면 문자 그대로 번역하여 의미가 달라질 수 있습니다.

어딜 도망가?

구글 ; Where are you running?

파파고 : Where are you going?

결론 : 둘 다 맞는 번역

한국어에서는 "도망간다" 라는 의미지만 상황에 따라

- 뛰어서 가는 것
- 어디로 가는 것

두 의미 모두 가능합니다.

두 번역 모두 자연스럽다고 설명합니다.

문맥에 따라 여러 번역이 모두 맞을 수 있으며, 반드시 하나의 정답만 존재하는 것은 아닙니다.

뭐하노?

구글 : What are you doing?

파파고 : What are you doing?

둘 다 맞는 번역으로 사투리에 대한 학습 진행

"뭐하노?" 는 경상도 사투리입니다.

표준어로는 뭐 하고 있니? 입니다.

두 번역기 모두 What are you doing? 으로 올바르게 번역하였습니다.

이는 번역기가 이미 사투리 데이터도 상당량 학습했음을 의미합니다.

최근의 신경망 번역기는 표준어뿐 아니라 사투리도 학습하여 비교적 자연스럽게 번역할 수 있습니다. 이는 대량의 자연어 데이터를 학습한 결과입니다.

「통촉하여 주시옵소서」를 비교합니다.

Google 번역 : Please contact me.

Papago : Please let me know.

두 번역기 모두 '통촉'이라는 고어(옛말)를 제대로 이해하지 못했습니다.

"통촉하다"는 상대방의 사정을 깊이 헤아려 달라는 뜻으로 사용되는 매우 오래된 표현입니다.

따라서 문자 그대로 번역하기보다 Please understand my situation. 또는

Please have mercy on me. 와 같이 상황에 맞는 의미로 번역해야 자연스럽습니다.

- 오래된 표현이나 고어는 학습 데이터가 부족하기 때문에 번역 품질이 낮아질 수 있습니다.
- 번역 모델은 단어뿐 아니라 시대적 표현도 충분히 학습해야 합니다.

2) 한글 번역기 부작용

(1) 해외 사례

- 한글 번역 실패 예



실제 해외에서 발생한 번역 오류 사례가 소개됩니다.

왼쪽에는 한국어 리뷰가 적혀 있고, 오른쪽에는 자동 번역 결과가 표시됩니다.

원래 한국어는 '이 스파 진짜 좋아요!' 와 같은 긍정적인 후기였지만, 자동 번역 과정에서 한자가 섞이거나 의미 없는 문자들이 출력되었습니다.

즉 번역기가

- 문장의 의미를 이해하지 못하고
- 문자 단위로 잘못 처리하여

전혀 다른 결과가 만들어진 사례입니다.

자동 번역기는 데이터가 부족하거나 문맥을 이해하지 못하면

- 의미 없는 문자
- 한자
- 깨진 문장

등을 출력할 수 있습니다.

(2) 국내 사례

영어 번역 실패 예



메뉴판 자동 번역 사례입니다. 번역기가 음식 이름을 의미가 아니라 발음 또는 단어 단위로 번역하여 웃긴 결과가 만들어졌습니다.

예를 들어, 설렁탕 : Bear Tang

"설"

↓

"Bear"

라고 잘못 번역했습니다.

육회 : Six Times

육(六)

↓

Six

회(回)

↓

Times

라고 문자 그대로 번역했습니다.

돈가스 정식 ; Money Gas Proper Form

돈

↓

Money

가스

↓

Gas

정식

↓

Proper Form

으로 각각 분리해서 번역한 사례입니다.

돼지 주물럭 : Massage Pork

주물럭

↓

Massage

라고 해석하여

실제 음식 이름을 전혀 표현하지 못했습니다.

기계 번역은

- 음식명
- 고유명사
- 전통 음식

처럼 사전에 없는 단어를 번역할 때 문자 단위로 해석하여 오류가 발생할 수 있습니다.

▪ “돼지 주물럭” 구글 번역

- Pork loin

돼지 부위는 맞으나 주물럭 부분의 번역을 놓침

▪ “돼지 주물럭” 네이버 파파고 번역

- Marinated Grilled Pork

정확함 / 자연어 데이터를 기반으로 학습 중

최근의 신경망 번역기는 과거보다 음식 이름까지도 훨씬 자연스럽게 번역합니다.

3) 구글 번역기의 발전

(1) PBMT 에서 GNMT 기술로 대전환

2006 년

- 구글 번역기 등장
- 초기에는 대중에게 번역 서비스를 제공하지 않음
 - UN 이 보유한 디지털 문서를 분류하고,
 - 외래어 번역을 위한 자연어 처리 서비스에 활용
- 초기 구글 번역기는 부자연스러운 번역 기술로 인해 비난받음

2006 년 구글 번역기가 처음 등장했을 당시에는 현재와 같은 인공지능 번역기가 아니었습니다.

주요 특징은 다음과 같습니다.

- 인터넷에 존재하는 문서를 통계적으로 분석
- 단어의 등장 빈도를 계산
- 가장 많이 사용되는 번역을 선택

즉, 문장의 의미를 이해하는 것이 아니라 **확률이 높은 번역을 선택하는 방식**이었습니다.

그래서

- 어순이 이상하고
- 조사 사용이 어색하며
- 긴 문장은 거의 번역하지 못했습니다.

2006 년 초기 구글 번역기는 **통계 기반(PBMT)** 번역을 사용했기 때문에 문맥을 이해하지 못했고 번역 품질도 높지 않았습니다.

PBMT (Phrase Based Machine Translation)

- 문장을 단어 또는 구(Phrase) 단위로 분리
- 가장 확률이 높은 번역 결과를 선택
- 문맥 이해가 어려움

PBMT 번역 과정은 다음과 같습니다.

한국어 문장

↓

단어 분리

↓

각 단어 번역

↓

다시 연결

예를 들어, 나는 학교에 간다. 를 번역하면

나는

↓

I

학교

↓

School

간다

↓

Go

처럼 각각 번역한 뒤 이어 붙입니다.

따라서

I school go

와 같이 부자연스러운 결과가 나올 수 있습니다.

PBMT 는 단어 순서를 충분히 고려하지 못하는 한계가 있습니다.

PBMT 는 문장을 전체적으로 이해하지 않고 **단어 또는 구 단위로** 번역하기 때문에 자연스러운 번역이 어렵습니다.

GNMT (Google Neural Machine Translation)

- 인공신경망 기반 번역
- 문장 전체 의미를 이해
- 번역 품질 향상

PBMT에서는

단어

↓

단어

↓

단어

순으로 번역했습니다.

반면 GNMT에서는

문장 전체

↓

Encoder

↓

의미 벡터

↓

Decoder

↓

번역 문장이라는 과정을 거칩니다.

즉, 문장을 먼저 이해한 후 번역을 수행합니다.

그래서

- 문맥
- 시제
- 조사
- 어순

까지 고려할 수 있습니다.

GNMT는 **Encoder-Decoder** 구조의 신경망을 이용하여 문장 전체 의미를 학습하는 번역 기술입니다.

GNMT 특징

- 긴 문장 번역 가능
- 문맥 이해
- 자연스러운 번역
- 번역 정확도 향상

GNMT 는 문장 전체를 하나의 의미 벡터로 변환한 후 번역을 수행하므로, 기존 PBMT 보다 훨씬 자연스러운 번역이 가능합니다. 이러한 구조는 이후 **Seq2Seq, Attention, Transformer** 모델의 발전으로 이어졌으며, 현재의 대규모 언어모델(LLM) 번역 기술의 기반이 되었습니다.

원문

A run for president is an attempt to be elected to become a president of a country.

번역 예 :

PBMT : 대통령에 대한 실행한 나라의 대통령으로 선출되고 시도되고 있다.

단어를 직역하다 보니 벌어진 실수

GNMT : 대통령 출마는 한 나라의 대통령이 되려는 시도입니다.

PBMT(통계 기반 번역) 와 **GNMT(신경망 번역)** 의 가장 큰 차이를 실제 문장을 이용하여 비교합니다.

여기서 **run for president** 는 대통령 선거에 출마하다 라는 하나의 관용 표현입니다.

그러나 PBMT 는 문장을 단어 단위로 해석하기 때문에 run 을 실행으로 번역해 버렸습니다.

결국 대통령에 대한 실행 이라는 이상한 문장이 만들어졌습니다.

반면 GNMT 는 문장 전체의 의미를 먼저 이해한 후 번역하기 때문에 대통령 출마는 한 나라의 대통령이 되려는 시도입니다. 처럼 자연스럽게 번역할 수 있습니다.

- PBMT 는 단어를 개별적으로 번역합니다.
- GNMT 는 문장 전체의 의미를 이해하여 번역합니다.
- 관용 표현이나 속어는 GNMT 가 훨씬 정확하게 번역합니다.

GNMT 특징

- 문맥(Context)을 이해
- 긴 문장의 번역 품질 향상
- 자연스러운 문장 생성
- 번역 정확도 향상

GNMT 는 단순한 단어 번역기가 아니라 **문장 생성 모델**에 가까운 번역 방식입니다. Encoder 가 문장의 의미를 학습하고 Decoder 가 새로운 언어의 문장을 생성하기 때문에 이전 세대보다 훨씬 자연스러운 번역이 가능합니다.

GNMT 의 성능 향상

- 문맥 이해 능력 향상
- 번역 정확도 향상
- 사람과 유사한 자연스러운 번역
- 다양한 언어 지원 확대

신경망 번역기는 대량의 번역 데이터를 학습하여

- 단어의 의미
- 문장의 의미
- 문맥
- 표현 방식

을 함께 학습합니다.

예를 들어 은행이라는 단어는 문맥에 따라 Bank 또는 River bank 으로 번역될 수 있습니다.

GNMT 는 주변 단어를 함께 고려하여 적절한 의미를 선택합니다.

이처럼 문맥 기반 번역이 가능해졌기 때문에 실제 사람이 번역한 것과 비슷한 품질을 제공할 수 있습니다.

GNMT 는 기존의 통계 기반 번역을 넘어 **딥러닝 기반 번역**으로 발전한 기술입니다. 문맥을 이해하고 자연스러운 문장을 생성할 수 있어 번역 품질이 크게 향상되었으며, 이후 **Seq2Seq**, **Attention**, **Transformer** 모델로 이어지는 현대 신경망 번역 기술의 기반이 되었습니다.

2. 구글 AI 번역 기술

1) GNMT 기술 소개

Google Neural Machine Translation

→ Recurrent Neural Network의 일종

GNMT는 Google Neural Machine Translation의 약자입니다. 기존 번역 방식처럼 단어를 따로따로 바꾸는 것이 아니라, RNN 계열 신경망을 사용하여 문장의 순서와 문맥을 함께 고려하는 번역 방식입니다.

- **Many-to-Many의 형태**
 - RNN 기반 Sequence-to-Sequence 방식
 - 입출력 모두 시계열 데이터(배열)
- **인코더 / 디코더 연결 구조**

구분	역할	예시
인코더	입력 문장	영어 문장
디코더	출력 문장	한글 문장

번역 : 영어 → 한글

번역 모델이 **인코더와 디코더**로 나뉘어 동작한다는 것을 보여줍니다.

인코더는 영어 문장을 입력받아 문장의 의미를 벡터로 압축합니다.

디코더는 이 압축 정보를 받아 한글 문장을 생성합니다.

즉, 번역은 단순 치환이 아니라

영어 문장 → 인코더 → 의미 정보 → 디코더 → 한글 문장

순서로 처리됩니다.

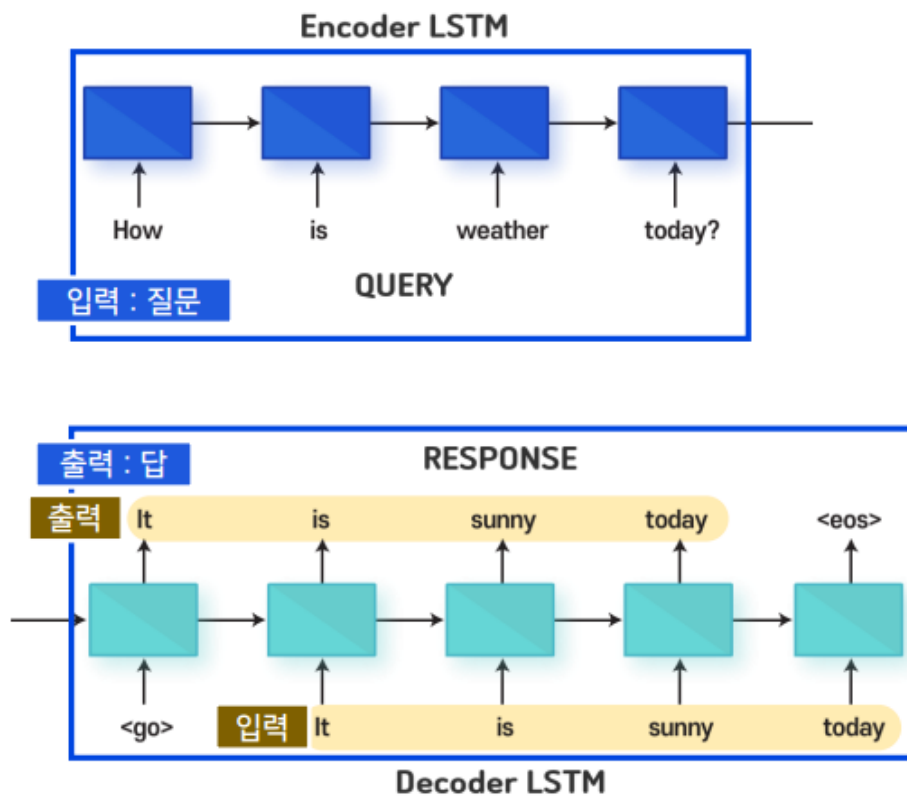
- **응용 분야**



GNMT와 같은 Sequence-to-Sequence 구조는 번역뿐 아니라 챗봇에도 사용될 수 있습니다. 번역에서는 입력 문장이 다른 언어의 출력 문장으로 바뀌고, 챗봇에서는 사용자의 질문이 답변 문장으로 바뀝니다.

핵심은 둘 다 **입력 시퀀스를 받아 출력 시퀀스를 생성한다**는 점입니다.

- 챗봇의 예



사용자가 질문을 입력하면 모델은 그 질문을 이해한 뒤 적절한 답변을 생성합니다.

예를 들어

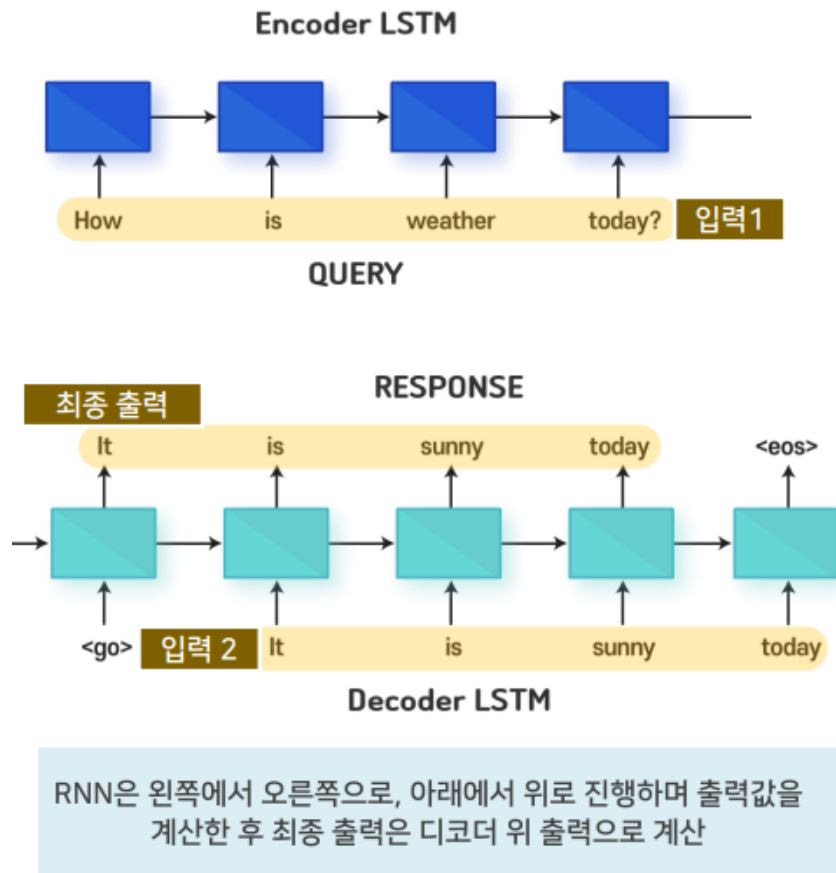
입력 : 오늘 날씨 어때?

출력 : 오늘은 맑습니다.

처럼 입력 문장과 출력 문장이 모두 순서를 가진 데이터입니다.

챗봇 구조에서 **입력 문장**이 먼저 RNN 또는 Seq2Seq 모델의 인코더에 전달됩니다.

모델은 입력 문장을 단어 또는 문자 단위로 순서대로 읽고, 질문의 의미를 내부 상태로 저장합니다.



챗봇은 이전 입력과 현재 입력을 함께 고려할 수 있습니다.

예를 들어 사용자가 여러 문장을 연속해서 입력하면, 모델은 앞선 문맥을 참고하여 최종 답변을 생성합니다.

즉, 단일 문장만 보는 것이 아니라 대화 흐름을 반영할 수 있습니다.

챗봇에서는 앞 문장과 뒤 문장의 관계가 중요합니다.

예를 들어 사용자가 “거기 위치는?”이라고 묻는다면, 이전 문장에서 말한 장소를 기억해야 정확한 답을 할 수 있습니다. 따라서 RNN 기반 모델은 순서가 있는 데이터를 처리하는 데 적합합니다.

디코더의 입력과 출력을 비교해서 지도학습을 진행하는 것이 Sequence-to-Sequence 기술의 핵심입니다. Seq2Seq 학습의 핵심은 **디코더 입력값**과 **디코더가 내야 할 정답 출력값**을 비교하는 것입니다.

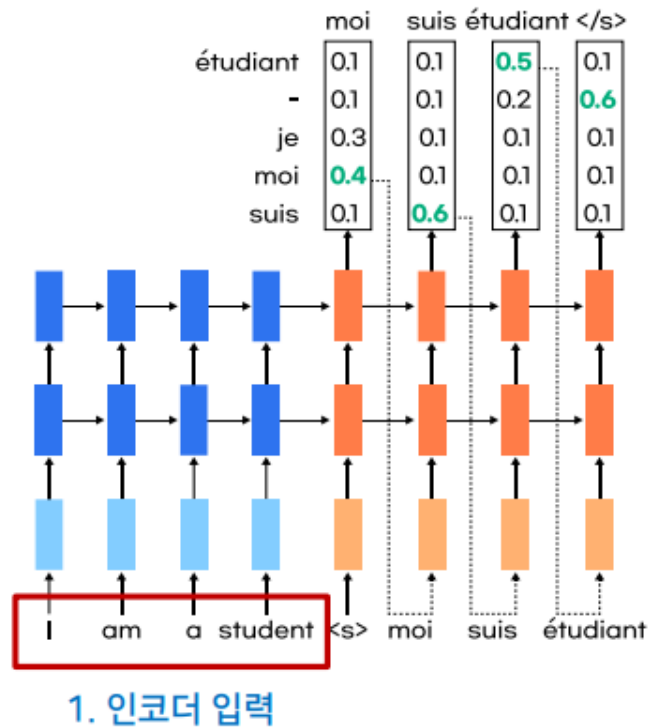
예를 들어 번역 정답이 ‘나는 학생이다’ 라면 디코더는 한 단어씩 다음 단어를 예측합니다.

모델이 예측한 단어와 실제 정답 단어를 비교하여 오차를 계산하고, 그 오차를 줄이도록 학습합니다.

2) GNMT 기술 개념도

(1) 영어를 불어로 번역

- 인코더 입력 : 번역을 원하는 영어 문장



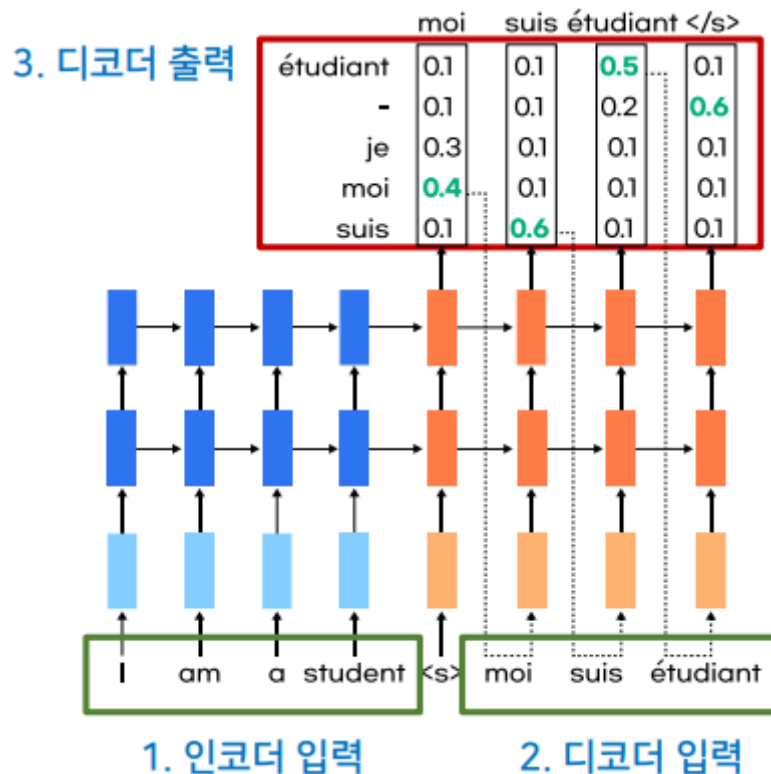
영어 문장을 프랑스어로 번역하는 Seq2Seq 구조의 첫 단계입니다.

인코더 입력은 번역하고 싶은 원문입니다.

예를 들어, 'I love you' 라는 영어 문장이 인코더에 입력됩니다.

인코더는 이 문장을 순서대로 읽고 문장의 의미를 내부 벡터로 압축합니다.

- 디코더 입력 : 불어 번역
 - 매 입력값마다 정답인 출력값 존재



디코더 입력은 번역 결과를 생성하기 위해 디코더에 넣는 문장 정보입니다.

학습 단계에서는 정답 프랑스어 문장을 한 칸씩 밀어서 디코더 입력으로 사용합니다.

예를 들어 정답이 'Je t'aime' 이라면 디코더는 시작 토큰 뒤에 차례대로 단어를 입력받고, 다음 단어를 맞히도록 학습합니다.

즉, 디코더는 매 시점마다 "다음에 와야 할 단어"를 예측합니다.

- 디코더 출력 : 왼쪽에서 오른쪽, 아래에서 위 방향으로 진행
- 각 단어의 출력값을 합하면 1(확률)

디코더 출력은 실제 모델이 예측한 번역 결과입니다.

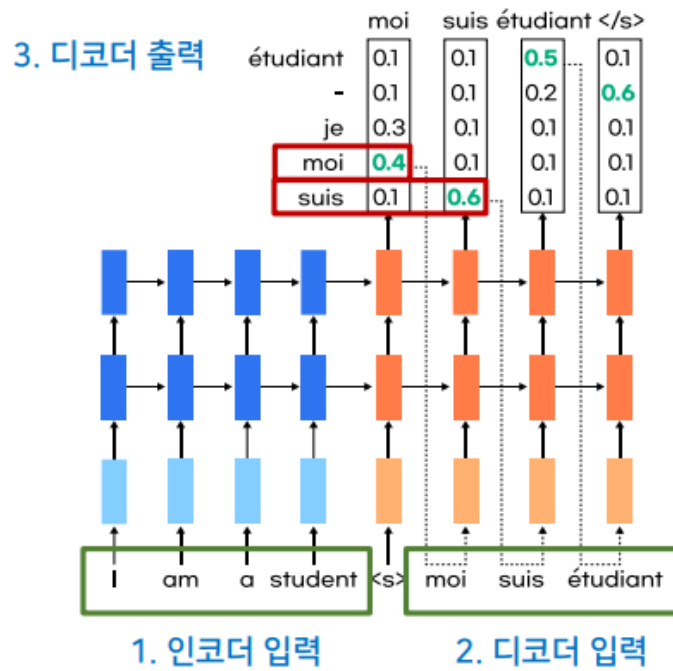
디코더는 한 번에 전체 문장을 출력하는 것이 아니라, 순서대로 단어를 하나씩 출력합니다.

예를 들어

Je → t' → aime

처럼 왼쪽에서 오른쪽으로 순차적으로 생성합니다.

이는 RNN이 시계열 데이터를 순서대로 처리하는 방식과 연결됩니다.



디코더는 각 위치에서 가능한 모든 단어에 대해 확률을 계산합니다.

예를 들어 첫 번째 출력 위치에서

Je : 0.80

Tu : 0.10

Il : 0.05

기타 : 0.05

처럼 여러 후보 단어의 확률을 계산합니다.

모든 후보 단어 확률을 더하면 1이 됩니다.

모델은 그중 가장 확률이 높은 단어를 선택하여 번역 결과로 사용합니다.

- 디코더 출력의 확률값을 이용하여 번역 결과를 선택
- 각 시점마다 가장 높은 확률의 단어를 선택
- 선택된 단어들이 모여 최종 번역 문장을 구성

디코더는 각 위치마다 여러 단어 후보를 출력합니다.

그중 가장 확률이 높은 단어를 고르면 번역 문장이 완성됩니다.

즉,

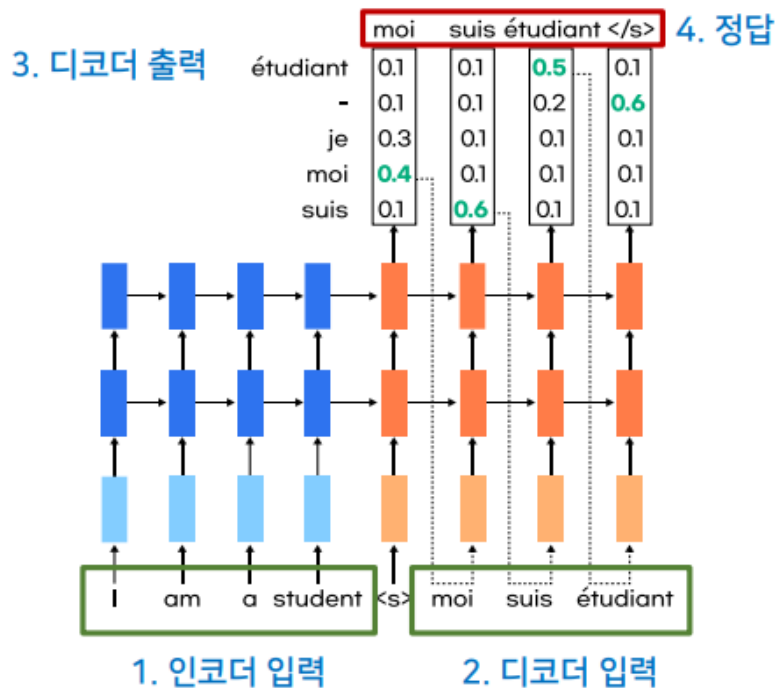
1번째 위치 → 가장 높은 확률 단어 선택

2번째 위치 → 가장 높은 확률 단어 선택

3번째 위치 → 가장 높은 확률 단어 선택

과정을 반복하여 최종 번역문을 만듭니다.

- 정답 : 모든 출력 단어를 확률 계산으로 처리하여 번역 결과 도출



구성은 다음과 같습니다.

1. 인코더 입력
번역할 원문 문장
2. 디코더 입력
정답 문장을 한 칸씩 밀어 넣은 입력
3. 디코더 출력
모델이 예측한 단어별 확률
4. 정답
실제 번역 문장

모델은 디코더 출력과 정답을 비교하면서 학습합니다.

예측이 정답과 다르면 손실이 커지고, 손실을 줄이는 방향으로 가중치가 수정됩니다.

따라서 Seq2Seq 번역 모델은 확률 기반 다중 분류 문제를 문장 길이만큼 반복하는 구조라고 볼 수 있습니다.

3. Seq2Seq

Seq2Seq 는 **Sequence-to-Sequence** 의 줄임말입니다. 즉, 하나의 순서 데이터(sequence)를 입력받아 다른 순서 데이터로 변환하는 딥러닝 구조입니다.

예를 들어 번역에서는 다음과 같습니다.

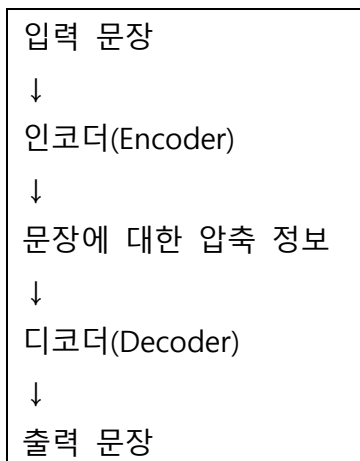
입력 시퀀스: 나는 당신을 사랑합니다

출력 시퀀스: I LOVE YOU

여기서 문장은 단어들이 순서대로 나열된 데이터입니다. 따라서 번역 모델은 단어의 의미뿐 아니라 **단어의 순서**도 함께 이해해야 합니다.

Seq2Seq 의 기본 구조

Seq2Seq 는 크게 두 부분으로 구성됩니다.



1. 인코더

인코더는 입력 문장을 읽는 부분입니다.

예를 들어

나는 / 당신을 / 사랑합니다

이라는 문장이 들어오면 인코더는 이 단어들을 순서대로 읽고, 문장 전체의 의미를 하나의 내부 정보로 압축합니다.

이 압축 정보는 보통 다음과 같은 상태값으로 표현됩니다.

Hidden State

Cell State

LSTM 을 사용하는 경우에는 이 두 상태가 디코더로 전달됩니다.

2. 디코더

디코더는 인코더가 만든 압축 정보를 바탕으로 새로운 문장을 생성합니다.

예를 들어 인코더가 한국어 문장의 의미를 압축했다면, 디코더는 이를 이용하여 영어 문장을 순서대로 생성합니다.

I → LOVE → YOU

디코더는 한 번에 전체 문장을 출력하는 것이 아니라, 매 시점마다 다음 단어를 예측합니다.

Seq2Seq 가 필요한 이유

일반적인 분류 모델은 입력 하나에 대해 결과 하나를 출력합니다.

예:

영화 리뷰 → 긍정 / 부정

이미지 → 고양이 / 강아지

하지만 번역은 입력과 출력의 길이가 다를 수 있습니다.

예:

나는 학생입니다.

→ I am a student.

입력은 2 개 단어 정도지만, 출력은 4 개 단어가 될 수 있습니다.

따라서 번역처럼 **입력 길이와 출력 길이가 다른 문제**에는 Seq2Seq 구조가 적합합니다.

번역 데이터 구성

실습 코드에서는 영어와 프랑스어 문장이 매핑된 데이터를 사용합니다.

데이터 예시는 다음과 같은 구조입니다.

src : 영어 문장

tar : 프랑스어 문장

실습에서는 `pd.read_csv()`를 사용하여 데이터를 읽고, 전체 샘플이 많으면 일부만 사용하여 학습 시간을 줄입니다.

SOS 와 EOS 토큰

Seq2Seq 에서는 문장의 시작과 끝을 알려주는 특수 토큰이 필요합니다.

SOS

SOS 는 **Start Of Sequence** 입니다.

문장이 시작된다는 의미입니다.

예: <sos> I love you

EOS

EOS 는 **End Of Sequence** 입니다.

문장이 끝났다는 의미입니다.

예: I love you <eos>

디코더 입력과 디코더 출력

Seq2Seq 학습에서 가장 중요한 부분은 디코더 입력과 디코더 출력입니다.

예를 들어 정답 문장이 다음과 같다고 하겠습니다.

I LOVE YOU

그러면 디코더 입력은 다음처럼 시작 토큰을 붙입니다.

<sos> I LOVE

디코더 출력은 다음처럼 끝 토큰을 붙입니다.

I LOVE YOU <eos>

즉, 디코더는 현재 단어를 보고 다음 단어를 맞히도록 학습합니다.

입력: <sos>	→	출력: I
입력: I	→	출력: LOVE
입력: LOVE	→	출력: YOU
입력: YOU	→	출력: <eos>

정수 인코딩

컴퓨터는 문자를 그대로 이해하지 못하므로 단어 또는 문자를 숫자로 바꾸어야 합니다.

예를 들어 문자 집합이 다음과 같다고 하면

I → 1
LOVE → 2
YOU → 3
<sos> → 4
<eos> → 5

문장은 다음처럼 숫자로 변환됩니다.

I LOVE YOU

→ [1, 2, 3]

이 과정을 **정수 인코딩**이라고 합니다.

패딩

문장마다 길이가 다르기 때문에 모델에 입력하기 위해 길이를 맞춰야 합니다.

예:

I love you → 길이 3

I am a student → 길이 4

길이를 4로 맞추면 다음과 같이 됩니다.

I love you <pad>

I am a student

이처럼 부족한 부분을 채우는 작업을 **패딩**이라고 합니다.

원-핫 인코딩

정수 인코딩된 값을 신경망 입력에 사용하기 위해 원-핫 인코딩을 적용할 수 있습니다.

예를 들어 단어 집합 크기가 5 라면

I → 1

은 다음처럼 표현됩니다.

[0, 1, 0, 0, 0]

하나의 위치만 1 이고 나머지는 0 인 벡터입니다.

RNN 기반 Seq2Seq 번역기 구조

RNN 기반 번역기는 다음 흐름으로 만들어집니다.

1. 데이터 읽기
2. 입력 문장과 출력 문장 분리
3. SOS, EOS 토큰 추가
4. 문자 또는 단어 집합 생성
5. 정수 인코딩
6. 패딩
7. 원-핫 인코딩
8. 인코더 LSTM 생성
9. 디코더 LSTM 생성
10. Dense 출력층 생성
11. 모델 학습
12. 번역 결과 예측

인코더 LSTM

인코더는 입력 문장을 읽고 마지막 상태값을 생성합니다.

Keras 기준으로는 다음 옵션이 중요합니다.

```
return_state=True
```

이 옵션을 사용하면 인코더의 최종 상태를 받을 수 있습니다.

인코더 출력 자체는 필요하지 않고, 보통 다음 두 값이 중요합니다.

state_h : 은닉 상태

state_c : 셀 상태

이 두 값이 디코더로 전달됩니다.

디코더 LSTM

디코더는 인코더에서 전달받은 상태값을 초기 상태로 사용합니다.

```
initial_state=[state_h, state_c]
```

또한 디코더는 문장의 모든 시점에서 단어를 예측해야 하므로 다음 옵션이 필요합니다.

```
return_sequences=True
```

이 옵션을 사용하면 디코더가 마지막 출력만 반환하지 않고, 모든 시점의 출력을 반환합니다.

Dense 출력층

디코더 LSTM의 출력은 Dense 층을 거쳐 최종 단어 확률로 변환됩니다.

예를 들어 프랑스어 단어 집합 크기가 7,000 개라면 매 시점마다 7,000 개 단어 중 하나를 선택해야 합니다.

즉, 디코더는 매 시점마다 다중 클래스 분류를 수행합니다.

현재 시점 출력 → Dense → 단어별 확률 → 가장 높은 확률의 단어 선택

손실 함수

디코더는 매 시점마다 단어 집합 중 하나를 맞혀야 하므로 다음과 같은 손실 함수를 사용합니다.

`categorical_crossentropy`

`sparse_categorical_crossentropy`

차이는 다음과 같습니다.

`categorical_crossentropy` → 정답이 원-핫 인코딩 형태일 때 사용

`sparse_categorical_crossentropy` → 정답이 정수 인덱스 형태일 때 사용

학습과 예측의 차이

학습 단계

학습할 때는 정답 문장을 알고 있습니다.

따라서 디코더 입력에 정답 문장의 앞부분을 넣고, 디코더 출력이 다음 단어를 맞히도록 학습합니다.

입력: <sos> I LOVE

정답: I LOVE YOU

이 방식은 모델이 더 안정적으로 학습하도록 도와줍니다.

예측 단계

예측할 때는 정답 문장이 없습니다.

따라서 디코더는 자신이 이전에 예측한 단어를 다시 다음 입력으로 사용합니다.

예:

1 단계: <sos> 입력 → I 예측
2 단계: I 입력 → LOVE 예측
3 단계: LOVE 입력 → YOU 예측
4 단계: YOU 입력 → <eos> 예측

<eos>가 나오면 문장 생성을 종료합니다.

Seq2Seq 는 입력 시퀀스를 출력 시퀀스로 변환하는 모델입니다.

번역에서는 다음과 같이 동작합니다.

입력 문장
→ 인코더가 의미 압축
→ 디코더가 번역 문장 생성

RNN 기반 Seq2Seq 에서는 LSTM 을 사용하여 문장의 순서를 기억하고, 인코더의 Hidden State 와 Cell State 를 디코더에 전달합니다.

디코더는 매 시점마다 단어 하나를 예측하며, 이 과정이 반복되어 최종 번역 문장이 생성됩니다.